A Straightforward Ridge-Regression Baseline for Heart-Disease Prediction

EPFL CS-433 - Project 1
Machine Learning, Fall 2026

***Abstract*—We construct a simple classifier for the given health survey dataset relying solely on NumPy and Python standard library. The pipeline consists of missing value imputation, feature standardization, and ridge regression. Given that 8.83% of the labeled dataset examples are assigned to the positive class, accuracy is not a sufficient metric to evaluate how well the model detects the minority class. In a 80/20 train/validation split, decreasing the decision threshold from 0.50 to 0.22 increases F1 from 0.0170 to 0.4140 and recall from 0.86% to 51.66% at the cost of increased false positives. Note that the threshold is adjusted using the same validation partition as that on which we report the results.**

I. Task and data

This task involves binary classification, i.e., predicting the heart disease label for a respondent based on their survey data. The dataset is provided with the course assignment and was constructed using the Behavioral Risk Factor Surveillance System (BRFSS) [1]. It does not contain any information about future risks and causalities related to disease.

The labeled dataset has 328,135 examples, 109,379 examples of the test set and 321 input features. Labels can take on the values of -1 (negative class) or +1 (positive class). In the labeled dataset, 28,975 (8.83%) examples are from the positive class. This is crucial because a simple classifier which always outputs -1 can achieve very high accuracy but detect no positives. Thus, we report precision, recall, and F1 along with accuracy.

The goal of this work is to provide a clear and easily reproducible baseline. We rely on no outside data, no feature interaction and no search through models.

II. Preprocessing

The preprocessing pipeline starts by splitting the identifier from the features and ensuring that training feature identifiers match label identifiers. Ids are retained for the submission while not used in the predictors. Features having empty or non-finite values are considered missing.

In validation, all the preprocessing statistics are computed on the training set alone without any influence of the validation set on the model building process. The steps followed include:

1) Drop columns with no finite training value.
2) Impute missing entries with the median of the column's finite training values.
3) Drop columns whose post-imputation standard deviation is at most 10^{-12}.
4) Center each remaining column by its training mean and scale by its training standard deviation, using NumPy's population standard deviation (`ddof=0`).
5) Append a constant one to enable an intercept.

This leaves 315 survey features plus the intercept. Validation rows reuse the retained columns, medians, means, and scales. For the final model, these statistics are re-estimated from all labeled rows, and 315 features are again retained.

III. Model and decision rule

We encode labels as targets $y_i \in \{0, 1\}$ and apply ridge regression. A linear model produces a score $s_i = x_i^\top w$, with x_i including the intercept. Here N is the number of fitting rows, X stacks their feature vectors, and w contains one weight per column. Ridge regression minimizes

$$J(w) = \frac{1}{2N} \sum_{i=1}^{N} (y_i - x_i^\top w)^2 + \lambda \|w\|_2^2. \tag{1}$$

The first term is half the mean squared prediction error; the second penalizes large weights and thereby limits model complexity. We set $\lambda = 0.001$ and solve

$$(X^\top X + 2N\lambda I)w = X^\top y \tag{2}$$

using `numpy.linalg.solve`. The intercept weight is included in the penalty, consistent with our implementation. If the system is singular, we fall back to a pseudoinverse.

The scores are neither calibrated probabilities nor confined to $[0, 1]$. To obtain a class, we output $+1$ when $s_i \geq t$ and -1 otherwise. Decreasing t generally increases the number of predicted positives, along with false positives.

IV. Experimental protocol

We permute all labeled row indices using `np.random.default_rng(42)`. The first 262,508 rows form the training split and the remaining 65,627 form the validation split. The split is random rather than stratified and contains 5,825 positive validation labels.

Table I
VALIDATION PERFORMANCE ON THE SAME 65,627 ROWS. ALL METRICS RANGE FROM ZERO TO ONE; HIGHER IS BETTER. THE SELECTED-THRESHOLD ROW IS EVALUATED ON ITS TUNING SPLIT.

Prediction rule	Accuracy	Precision	Recall	F1
Always negative	0.9112	0.0000	0.0000	0.0000
Ridge, $t = 0.50$	0.9118	0.8197	0.0086	0.0170
Ridge, $t = 0.22$	0.8702	0.3455	0.5166	0.4140

We fit a single ridge model and compare three prediction rules: always negative, ridge with the conventional threshold $t = 0.50$, and ridge with a threshold chosen to maximize validation F1. We scan 46 thresholds from 0.05 to 0.50 in increments of 0.01 and pick the smallest threshold achieving the highest F1. No other hyperparameter tuning is performed.

For the positive class, TP denotes correctly detected positives, FP denotes negatives predicted positive, and FN denotes missed positives. The metrics are

$$\begin{aligned} \text{Precision} &= \frac{\text{TP}}{\text{TP} + \text{FP}}, \\ \text{Recall} &= \frac{\text{TP}}{\text{TP} + \text{FN}}, \\ \text{F1} &= \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}. \end{aligned} \tag{3}$$

Accuracy is the fraction of all predictions that are correct. For an all-negative classifier, precision is undefined and recorded as zero.

V. Results and interpretation

Table I lists the measured validation results. Threshold selection yields $t = 0.22$.

To begin with, accuracy alone will not work in this case. The always-majority baseline achieves 91.12% accuracy yet 0% recall and 0% F1 score, while Ridge classifier with $t = 0.50$ obtains a very similar accuracy and under 1% of positives.

The second observation is that the decision threshold matters a lot. With both model weights and preprocessing parameters fixed, decreasing the threshold value to 0.22 improves the F1 score by 0.3970 and recall by approximately 50.80 percentage points. The model now detects almost half of all positives, but the precision rate is only 34.55%—approximately 1 in 3 predictions of positives are actually positive. Overall accuracy decreases to 87.02%.

In this comparison, the threshold effect was separated from others. We did not run any ablation experiments to separate effects of imputation, scaling, and regularization.

VI. Reproducibility and implementation

The measured run used Python 3.14.6 and NumPy 2.5.3. The code needs NumPy 1.23 or later and no external ML libraries. From the project directory, with the supplied archive at `data/dataset_to_release.zip`, run:

```
python run.py
```

Alternatively, place `x_train.csv`, `y_train.csv`, and `x_test.csv` in `data/dataset/`. Use `--data-dir` to point to another folder. Full validation metrics are written to `submission.metrics.json` [2].

After threshold selection, the script refits preprocessing and ridge regression on all 328,135 labeled rows, keeps $t = 0.22$, and writes `submission.csv` containing the original test IDs and 109,379 predictions in $\{-1, +1\}$. Test labels are unavailable locally, and no competition score is reported.

The accompanying `implementations.py` supplies all six required methods: MSE gradient descent, MSE stochastic gradient descent with batch size one, least squares, ridge regression, logistic regression, and regularized logistic regression. Only ridge is evaluated in this pipeline. Each function returns the final weights and the unpenalized loss. All 27 applicable supplied public tests passed locally; the GitHub URL format test was skipped because a local project directory was used.

VII. Limitations and conclusion

Since the chosen threshold is validated on the same split used to choose it, it is subject to optimistic bias and we would need to use a holdout or nested cross-validation for an unbiased assessment. A single split result with no confidence intervals is reported.

Numeric codes of categorical values are treated as regular numeric values without any survey-specific refusal and unknown codes interpretation. No one-hot encodings and other feature engineering are performed. This simplicity allows having a transparent baseline, yet losing some information.

The major observation is that class imbalance affects the way the model performance has to be assessed: the accuracy may be high despite of the lack of positive detections. The simple ridge regression model with a modified threshold performs significantly better than the default one on the validation set F1, yet produces many errors.

References

[1] EPFL CS-433, *Class Project 1*, Machine Learning course assignment, Fall 2026. Supplied `project1_description.pdf`.

[2] Project experiment artifacts: `run.py`, `implementations.py`, and `submission.metrics.json`, generated with seed 42.